

CPE 486/586: Deep Learning for Engineering Applications

06 Generative Adversarial Network

Spring 2026

Rahul Bhadani

Outline

- 1. Motivation**
- 2. Generative Adversarial Networks**
- 3. GANs (in PyTorch)**
- 4. Examples**
- 5. Challenges with GANs**
- 6. Advanced Architecture in GAN**
 - 6.1 Deep Convolution GAN
 - 6.2 Cycle GAN
 - 6.3 Wasserstein GAN
- 7. Data Privacy and GANs**

Lecture Overview

- ⚡ In this lecture, we discussed generative models (briefly) and how to model distributions of data using density estimation techniques. One advantage of these generative models is that we can generate data from the distribution once we have the model.
- ⚡ Our goal in this lecture is to introduce *Generative Adversarial Networks* (GANs) for generating data from a distribution $p_{data}(\mathbf{x})$. GANs are able to model complex distributions of data that would be incredibly challenging to model using the density estimation approaches discussed in previous lectures.
- ⚡ Modeling $p_{data}(\mathbf{x})$ is increasingly challenging for complex data, such as real scenes, people, etc.

Motivation

1	0	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0	1	0
1	1	1	1	0	0	0	0	1	1	1	1	1	1	0	1	1	1	0	0
1	0	0	1	0	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0

Discriminative vs. Generative Models

Discriminative

Focuses on the **boundary** between classes.

- ⚡ Maps high-dimensional input $\mathbf{x}$ to label y .
- ⚡ Learns $P(y | \mathbf{x})$.
- ⚡ Goal: Classification and Regression.

Generative

Focuses on the **distribution** of the data.

- ⚡ Models the process that *creates* the data.
- ⚡ Learns $P(\mathbf{x}, y)$ or $P(\mathbf{x})$.
- ⚡ Strategy: MLE or Variational Inference.

While **Discriminative** models tell you how to tell things apart, **Generative** models give you the "recipe" for the data itself.

GAN is a generative model.

Adversarial Machine Learning

Wikipedia Definition

Study of attacks on ML algorithms and defenses against such attacks.

- ⚡ **Data Assumption:** Conventional training is based on the IID assumption; however, someone can supply non-IID data.
- ⚡ **Exploitation:** Adversaries may attempt to exploit a learning mechanism either to cause it to misbehave or to extract or misuse information.



While we won't delve into Secure Machine Learning, we serve this as a motivation for **Generative Adversarial Network (GAN)**.

GAN is a **two-player minimax** game where:

Generator: A counterfeiter (adversary) trying to fool you.

Discriminator: A police officer investigating and detecting counterfeit objects.

Example of a Complex $p_{data}(\mathbf{x})$



Example of a Complex $p_{data}(\mathbf{x})$



Example of a Complex $p_{data}(\mathbf{x})$



Example of a Complex $p_{data}(\mathbf{x})$



None of these people are real!



There are hundreds(?) of different types of GANs; however, this lecture only presents the vanilla GAN.

Generative Adversarial Networks

0	0	1	1	0	0	0	1	1	1	1	0	0	1	1	0	1	0	1	1
1	0	1	1	1	0	1	1	1	0	1	0	0	1	0	1	1	0	1	1
0	1	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	1	1	0

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- ⚡ Access to $\{\mathbf{x}_i\}_{i=1}^n$
- ⚡ No labels needed (not classification).

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- ⚡ Access to $\{\mathbf{x}_i\}_{i=1}^n$
- ⚡ No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- ⚡ Access to $\{\mathbf{x}_i\}_{i=1}^n$
- ⚡ No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a “fake” data sample that is generated from $\mathbf{z}$.

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- ⚡ Access to $\{\mathbf{x}_i\}_{i=1}^n$
- ⚡ No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a “fake” data sample that is generated from $\mathbf{z}$.

Discriminator $D(\mathbf{x})$

D takes vector $\mathbf{x}$ and tries to discern if $\mathbf{x}$ was sampled from $p_{data}(\mathbf{x})$ or generated by G .

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- ⚡ Access to $\{\mathbf{x}_i\}_{i=1}^n$
- ⚡ No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

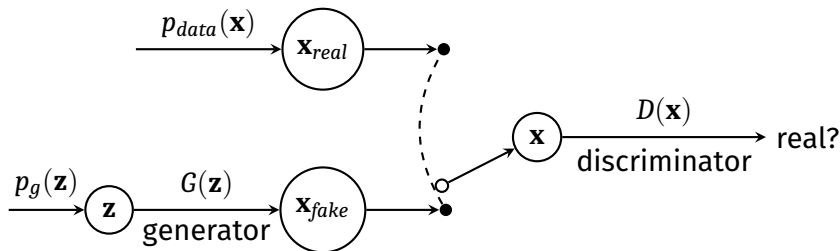
G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a “fake” data sample that is generated from $\mathbf{z}$.

Discriminator $D(\mathbf{x})$

D takes vector $\mathbf{x}$ and tries to discern if $\mathbf{x}$ was sampled from $p_{data}(\mathbf{x})$ or generated by G .

Let θ_g and θ_d represent the parameters for the generator and discriminator neural networks, respectively.

Generative Adversarial Net



Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative adversarial nets." *Advances in neural information processing systems* 27 (2014).

Generative Adversarial Net

⚡ We have two neural networks in a GAN

- The network G 's goal is to generate samples that look like they came from $p_{data}(\mathbf{x})$.
- The network D 's goal is to effectively discriminate between real samples from $p_{data}(\mathbf{x})$ and fake samples from G .

⚡ The goals of G and D are different! We can view this scenario as a game between D and G .

We train D to maximize the probability of assigning the correct label to both training examples and samples from G . We simultaneously train G to minimize $\log(1 - D(G(z)))$.

Formally Defining the Optimization Task

The optimization of θ_g and θ_d can be cast as a min-max task, which is given by

$$\min_G \max_D V(D, G) = \{\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log(D(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))]\} \quad (1)$$

V is called the value function.

From the original paper: alternate between k steps of optimizing D and one step of optimizing G .

In the beginning, G is poor, so D will reject samples with a high confidence, and $\log(1 - D(G(z)))$ saturates. So, rather than training G to minimize $\log(1 - D(G(z)))$, G is trained to maximize $\log D(G(z))$.

Formally Defining the Optimization Task

- ⚡ If $\mathbf{x} \sim p_{data}(\mathbf{x})$ then the discriminator should maximize $\log D(\mathbf{x})$
- ⚡ If $\mathbf{z} \sim p_g(\mathbf{z})$ then the generator seeks to generate a sample that makes the discriminator think it is a normal sample.
- ⚡ We can use stochastic gradient descent and ascent to adjust the parameters of the network.

Global Optimality of $p_g = p_{\text{data}}$

If we fix the generator G , the optimal discriminator D is

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$

Minimax Equation with Optimal Discriminator

For a fixed G , the minimax is

$$\begin{aligned} C(G) &= \max_D V(D, G) \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log(D_G^*(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D_G^*(G(\mathbf{z})))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log(D_G^*(\mathbf{x}))] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] \end{aligned}$$

Here, the training objective of D can be interpreted as maximizing the log-likelihood for estimating the conditional probability $P(Y = y|\mathbf{x})$ where Y is the random variable denoting where $\mathbf{x}$ comes from p_{data} with $y = 1$ or from p_g with $y = 0$.

Formally Defining the Optimization Task

Theorem 1. The global minimum of the virtual training criterion $C(G)$ is achieved if and only if $p_g = p_{data}$. At that point, $C(G)$ achieves the value $-\log(4)$. This point is a Nash Equilibrium.

Pseudo Code for Training a GAN

for $t = 1, \dots, T$

for $k = 1, \dots, K$

- Sample m noise data $\mathbf{z}_1, \dots, \mathbf{z}_m$ from $p_g(\mathbf{z})$
- Sample m data samples $\mathbf{x}_1, \dots, \mathbf{x}_m$ from $p_{data}(\mathbf{x})$
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \left\{ \frac{1}{m} \sum_{i=1}^m [\log(D(\mathbf{x}_i)) + \log(1 - D(G(\mathbf{z}_i)))] \right\} \quad (2)$$

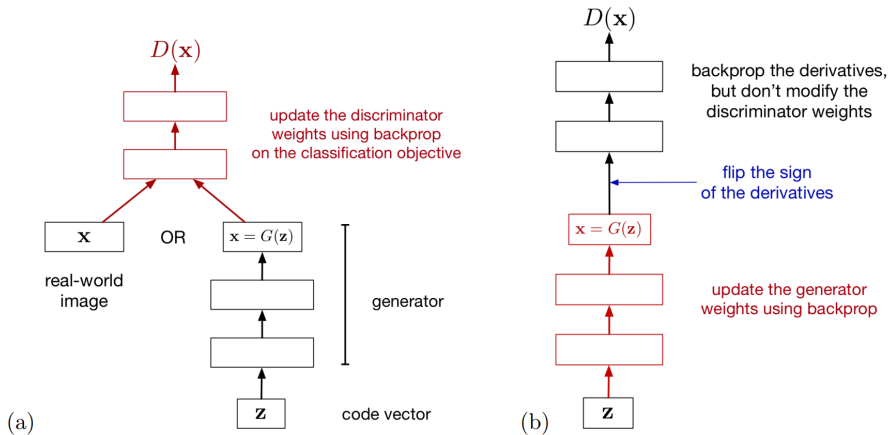
end for

- Sample m noise data $\mathbf{z}_1, \dots, \mathbf{z}_m$ from $p_g(\mathbf{z})$
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \left\{ \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}_i))) \right\} \quad (3)$$

end for

Formally Defining the Optimization Task



From Roger Grosse's GAN lecture notes.

On Training GANs

Challenge

One issue associated with training a GAN is the form of the loss. Observe above that the generator weights only get updated by the gradients of the second term:

$$\log(1 - D(G(\mathbf{z}))) \quad (4)$$

- ⚡ The problem with this is that if the generator sample is really bad, then the discriminator's prediction is close to zero, and since $\log(1 - D)$ is very flat when $D \approx 0$, there is not enough "signal" to move the generator weights meaningfully.
- ⚡ Increasing learning rates does not seem to help. This is called the *saturation problem* in GANs.
- ⚡ To fix this, while updating generator weights, it is common to heuristically replace the second term in the GAN loss with $-\log D(G(\mathbf{z}))$

On Training GANs

Challenge

Stably training GANs was a challenge faced by the community for quite some time (and continues to be a challenge), and a common resolution is to use *Wasserstein GANs*.

- ⚡ The GAN loss is a form of distance between probability distributions (called the Jensen-Shannon divergence), and this can be generalized to other distances.
- ⚡ A common alternative is the Earth-mover or Wasserstein distance, leading to a different type of GAN model called Wasserstein GAN. The WGAN was shown to improve stability of the optimization process

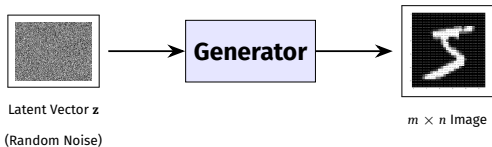
GANs (in PyTorch)

1	1	0	1	1	0	0	0	0	1	1	1	1	1	1	0	0	1	1	1
1	1	0	1	1	0	1	1	1	0	0	0	1	1	0	0	1	1	0	0
0	1	1	0	1	1	1	0	0	1	1	0	1	1	1	0	0	1	0	0

Generators

Conceptual Flow

The Generator takes random noise (latent vector z) and generates an $m \times n$ image.



PyTorch Implementation

```
1 class Generator(nn.Module):
2     def __init__(self, latent_dim=100, height=28, width=28):
3         super(Generator, self).__init__()
4         self.latent_dim = latent_dim
5         self.height, self.width = height, width
6
7     self.model = nn.Sequential(
8         nn.Linear(latent_dim, 256),
9         nn.LeakyReLU(0.2),
10        nn.Linear(256, 512),
11        nn.LeakyReLU(0.2),
12        nn.Linear(512, 1024),
13        nn.LeakyReLU(0.2),
14        nn.Linear(1024, width*height),
15        nn.Tanh() # Output in [-1, 1]
16    )
17
18    def forward(self, z):
19        img = self.model(z)
20        # Reshape to [batch, channels, H, W]
21        return img.view(img.size(0), 1,
22                        self.height, self.width)
```

Discriminators

Conceptual Flow

The discriminator takes a generated image and guesses whether it is fake or real through binary classification.



PyTorch Implementation

```
1 class Discriminator(nn.Module):
2     def __init__(self, height = 28, width = 28):
3         super(Discriminator, self).__init__()
4         self.height, self.width = height, width
5
6         self.model = nn.Sequential(
7             nn.Linear(self.height*self.width, 1024),
8             nn.LeakyReLU(0.2),
9             nn.Dropout(0.3), # Regularization
10            nn.Linear(1024, 512),
11            nn.LeakyReLU(0.2),
12            nn.Dropout(0.3),
13            nn.Linear(512, 256),
14            nn.LeakyReLU(0.2),
15            nn.Dropout(0.3),
16            nn.Linear(256, 1),
17            nn.Sigmoid() # Output probability [0, 1]
18        )
19
20    def forward(self, img):
21        img_flat = img.view(img.size(0), -1) # Flatten
22        validity = self.model(img_flat)
23        return validity
```

Training Loss in GAN Implementation

Training loss is actually just Binary Cross-Entropy loss effectively.

$$\mathcal{L}_{BCE}(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Case 1: Real Sample ($y = 1$)

$$\mathcal{L}_{BCE} = -\log D(x)$$

Case 2: Fake Sample ($y = 0$)

$$\mathcal{L}_{BCE} = -\log(1 - D(G(z)))$$

Exactly the negative of the value function mentioned in the original paper.

We also noted earlier that GoodFellow recommended to maximize $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$, because early training: D easily detects fakes $\Rightarrow D(G(z)) \approx 0$

$$\frac{\partial}{\partial \theta} \log(1 - D(G(z))) \approx 0 \quad \textbf{(Vanishing gradients!)}$$

Theory (Maximize)

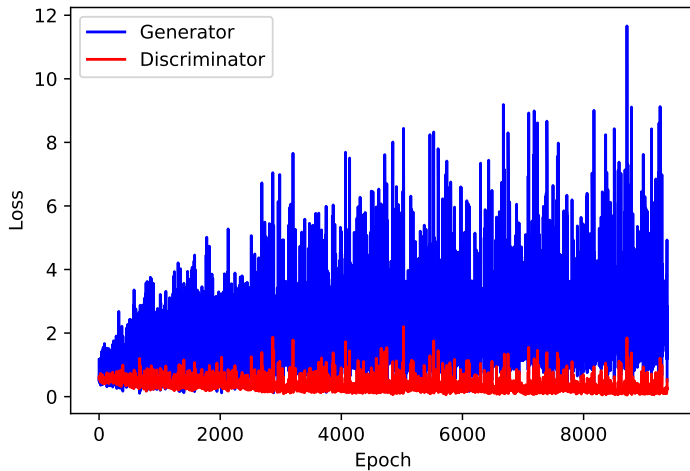
$$\begin{aligned} &\mathbb{E}[\log D(x)] \\ &\mathbb{E}[\log(1 - D(G(z)))] \end{aligned}$$

Code (Minimize)

$$\begin{aligned} &\text{criterion}(D(x), 1) \\ &\text{criterion}(D(G(z)), 0) \end{aligned}$$

Implementation: Just flip the label!

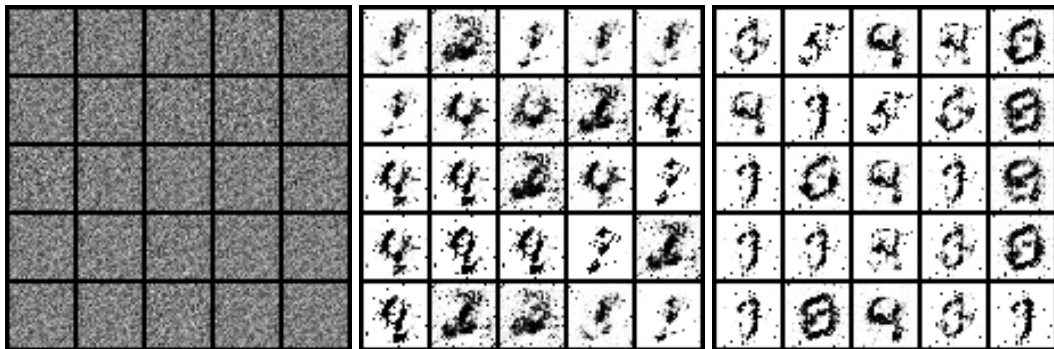
GAN Loss



Examples

1	0	0	0	0	1	0	1	1	0	0	1	0	1	1	1	1	1	0	0
1	1	1	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	1	1
1	0	1	0	0	1	0	0	0	0	1	1	1	0	1	0	0	1	1	0

GAN example



GAN example: Sketch to Figure



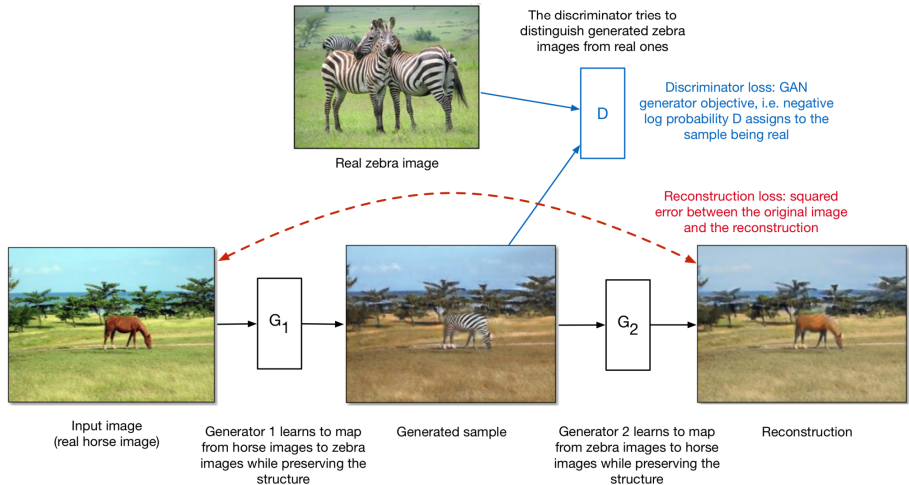
GAN example: Cycle GANs

- ⚡ Image-to-image translation involves generating a new synthetic version of a given image with a specific modification, such as translating a summer landscape to winter

GAN example: Cycle GANs

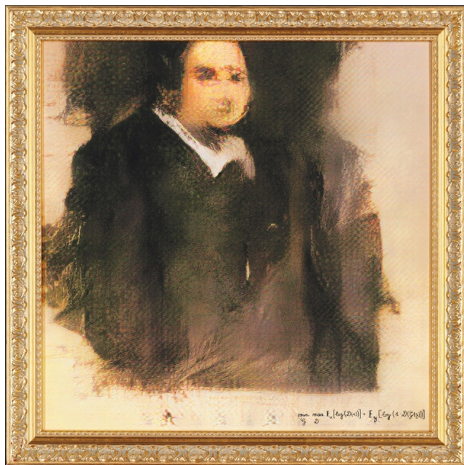


GAN example: Cycle GANs



$$\text{Total loss} = \text{discriminator loss} + \text{reconstruction loss}$$

First AI-Generated Portrait Ever Sold at Auction Shatters Expectations: \$432,500



Cosmopolitan Meets DALL-E 2



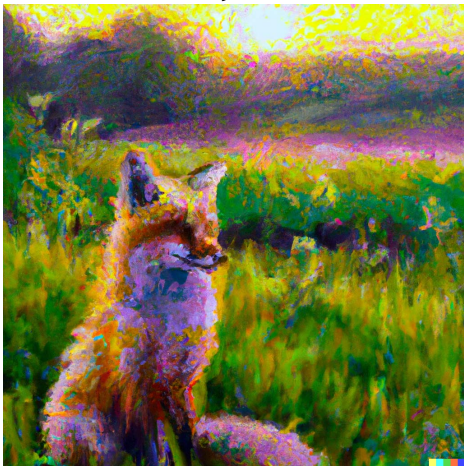
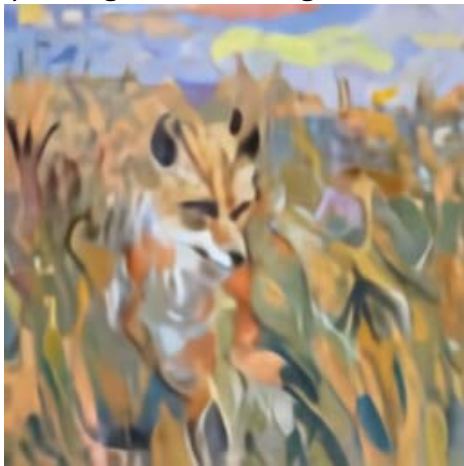
DALL-E 2: Text-to-Image Synthesis

- ⚡ DALL-E 2 is a new AI system that can create realistic images and art from a description in natural language. DALL-E 2 can create original, realistic images and art from a text description. It can combine concepts, attributes, and styles.
- ⚡ DALL-E uses a multimodal version of GPT-3 with 12 billion parameters. The concept is that it swaps text for pixels, and it is trained on text-image pairs from the Internet. DALL-E 2 uses 3.5 billion parameters.

DALL-E 2

“a painting of a fox sitting in a field at sunrise in the style of Claude Monet”

“a painting of a fox sitting in a field at sunrise in the style of Claude Monet”



Challenges with GANs

0 0 1 1 1 0 1 1 1 1 1 1 0 1 1 1 1 1 0
1 1 0 1 1 1 0 1 0 1 0 1 0 1 1 0 1 1 1
1 1 1 1 1 1 0 0 1 0 0 0 1 1 0 0 0 1 1

Issues with GANs (Google's Comments)

- ⚡ Research has suggested that if your discriminator is too good, then generator training can fail due to vanishing gradients. In effect, an optimal discriminator doesn't provide enough information for the generator to make progress.
 - The Wasserstein loss is designed to prevent vanishing gradients even when you train the discriminator to optimality.
- ⚡ The generator may learn to produce a single output. The generator is always trying to find the one output that seems most plausible to the discriminator. This form of GAN failure is called *mode collapse*.
- ⚡ GANs frequently fail to converge, as discussed in the module on training.

Advanced Architecture in GAN

1	1	0	0	0	1	0	0	0	1	0	1	0	1	0	1	1	0	1	1
0	0	1	1	1	1	1	0	0	0	0	0	1	0	1	1	1	0	0	0
1	1	1	1	0	0	1	0	1	0	1	0	0	0	0	1	0	0	1	1

Advanced Architecture in GAN

Deep Convolution GAN



1	1	1	0	0	0	0	1	1	0	1	0	0	0	1
0	1	1	1	1	0	0	1	0	1	0	0	1	1	1

Deep Convolution GAN

Fractionally-Strided Convolutional Layer (or Transposed Convolution) is the opposite of the convolution operation.

Key Concepts:

- ⚡ **Standard Convolution:** In a convolution operation (for example, stride = 2), we obtain a downsampled (smaller) output of the larger input.
- ⚡ **Fractional Stride:** In fractionally-strided convolution, an upsampled (larger) output is obtained from a smaller input.

Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." arXiv preprint arXiv:1511.06434 (2015).

Mathematical Example: Consider $X = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix}$ and filter as $K = \begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}$

We insert zeros between every other input pixel. Hence:

$$X = \begin{bmatrix} x_{00} & 0 & x_{01} & 0 \\ 0 & 0 & 0 & 0 \\ x_{10} & 0 & x_{11} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Then we apply a regular convolution operation.

Deep Convolution GAN

From the original paper:

- 1 Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- 2 Use batchnorm in both the generator and the discriminator.
- 3 Remove fully connected hidden layers for deeper architectures.
- 4 Use ReLU activation in the generator for all layers except for the output, which uses Tanh.
- 5 Use LeakyReLU activation in the discriminator for all layers.

The DCGAN made possible vector arithmetic for image manipulation: $\text{vector("King")} - \text{vector("Man")} + \text{vector("Woman")} = \text{vector("Queen")}$.

Advanced Architecture in GAN

Cycle GAN



0	1	1	1	0	0	0	0	0	0	0	1	1	0	1
0	1	0	0	0	1	1	0	1	0	1	0	1	1	1

Cycle GAN Architecture

It is used for image-to-image translation to transfer some interesting features from one image to another image, as we saw earlier. Examples: Horse to Zebra, Summer to Winter, etc.

Cycle GAN introduces **Cycle Consistency Loss** in addition to adversarial Loss seen in the original GAN.

Cycle GAN has four networks:

- 1 $G_{A \rightarrow B}$: a generator for translating images from domain $A \rightarrow B$.
- 2 D_B : a discriminator of whether an image is real or fake in B .
- 3 $G_{B \rightarrow A}$: a generator for translating images from domain $B \rightarrow A$.
- 4 D_A : a discriminator if an image is real or fake in A .

Zhu, Jun-Yan, Taesung Park, Phillip Isola, and Alexei A. Efros. "Unpaired image-to-image translation using cycle-consistent adversarial networks." In Proceedings of the IEEE International Conference on Computer Vision, pp. 2223-2232. 2017. <https://arxiv.org/pdf/1703.10593>

Cycle GAN Problem Statement

Domain Definition

- ⚡ X : Source domain
- ⚡ Y : Target domain

Mapping Objective:

Find a mapping, $G : X \rightarrow Y$, such that the distribution of images from $G(X)$ is indistinguishable from Y using an adversarial loss.

- ⚡ Output $\hat{y} = G(x)$, $x \in X$
- ⚡ Indistinguishable from $y \in Y$ by an adversary.

Basically, we need output distribution over $\hat{y}$ matching empirical distribution $p_{data}(y)$.

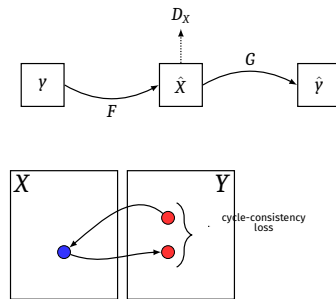
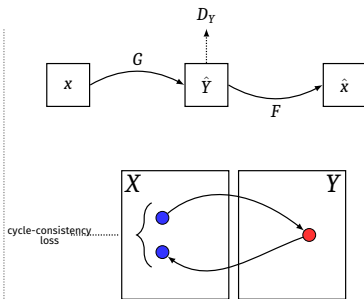
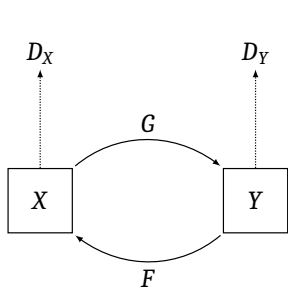
Dual Mapping

Alternatively, we can learn $F : Y \rightarrow X$.

Using a new kind of loss **cycle consistency loss** to enforce:

$$F(G(X)) \approx X$$

Cycle consistency means that if we translate from English to French and then back from French to English, we should get the same sentence.



Cycle consistency in domain-to-domain mapping for image style transfer

Loss Functions in CycleGAN

Adversarial Loss

Adversarial Loss for $G : X \rightarrow Y$:

$$\mathcal{L}_{GAN}(G, D_Y, X, Y) =$$

$$\mathbb{E}_{y \sim p_Y} [\log D_Y(y)] + \mathbb{E}_{x \sim p_X} [\log(1 - D_Y(G(x)))]$$

Adversarial Loss for $F : Y \rightarrow X$:

$$\mathcal{L}_{GAN}(F, D_X, Y, X) =$$

$$\mathbb{E}_{x \sim p_X} [\log D_X(x)] + \mathbb{E}_{y \sim p_Y} [\log(1 - D_X(F(y)))]$$

Cycle Consistency Loss

As our goal in cycle consistency is $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$, we have forward cycle consistency loss. Similarly, a backward cycle consistency requires $y \rightarrow F(y) \rightarrow G(F(y)) \approx y$.

Cycle consistency loss is

$$\mathcal{L}_{cyc}(G, F) =$$

$$\mathbb{E}_{x \sim p_X} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_Y} [\|G(F(y)) - y\|_1]$$

It captures differences in pixel values among generated and original images.

Full loss function is $\mathcal{L}(G, F, D_X, D_Y) = \mathcal{L}_{GAN}(G, D_Y, X, Y) + \mathcal{L}_{GAN}(F, D_X, Y, X) + \lambda \mathcal{L}_{cyc}(G, F)$ and the objective is

$$G^*, F^* = \arg \min_{G, F} \max_{D_X, D_Y} \mathcal{L}(G, F, D_X, D_Y).$$

Network Architecture

Generator:

- 1 Three convolutions, several residual blocks
- 2 Fractionally-strided convolution with stride of $\frac{1}{2}$
- 3 One convolution to map features to RGB.
- 4 6 blocks for 128×128 images, and 9 blocks for 256×256 or higher resolution images.
- 5 Use normalization

Discriminator:

- 1 Use 70×70 PatchGANs to classify whether 70×70 overlapping image patches are real or fake.
- 2 The PatchGAN discriminator is a fully convolutional network (FCN) with 4 layers of 4×4 convolutions (stride 2) — no fully connected (FC) layers.
- 3 Patch-level discriminator architecture has fewer parameters than a full-image discriminator and can work on arbitrarily sized images in a fully convolutional fashion

Replacing Negative Log-likelihood with Least Squares Loss

- 1 Train G to minimize $\mathbb{E}_{x \sim p_{\text{data}}(x)} [(D(G(x)) - 1)^2]$
- 2 Train D to minimize $\mathbb{E}_{y \sim p_{\text{data}}(y)} [(D(y) - 1)^2] + \mathbb{E}_{x \sim p_{\text{data}}(x)} [(D(G(x)))^2]$

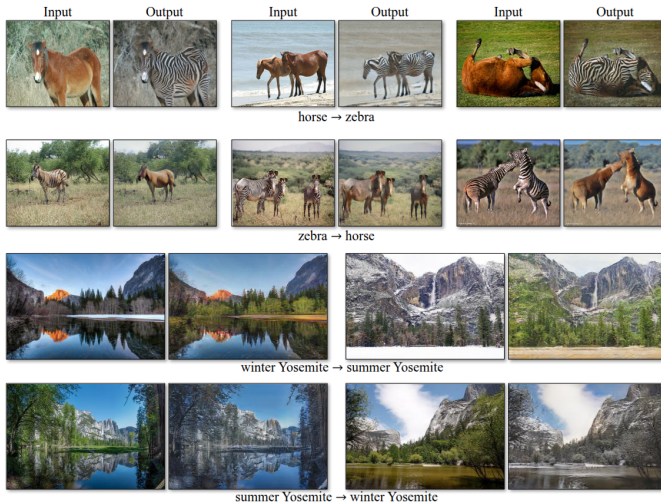
Reduce Model Training Oscillation

- 1 Update discriminators using the history of images rather than the one just produced by the latest generators. A buffer of the last 50 images is kept.
- 2 $\lambda = 10$, learning rate $\eta = 0.0002$.
- 3 Gaussian weight distribution initialization $\mathcal{N}(0, 0.02)$.

Dataset Used for CycleGAN evaluation

- 1 Maps $\longleftrightarrow$ Aerial Photos
- 2 Architectural facades labels $\longleftrightarrow$ photo
- 3 Winter $\longleftrightarrow$ Summer
- 4 Painting $\longleftrightarrow$ Photo
- 5 Cityscapes label $\longleftrightarrow$ Photo
- 6 Horse $\longleftrightarrow$ Zebra
- 7 Apple $\longleftrightarrow$ Orange

Results



Wasserstein GAN

Arjovsky, Martin, Soumith Chintala, and Léon Bottou. "Wasserstein GAN." arXiv preprint arXiv:1701.07875 (2017).

<https://arxiv.org/abs/1701.07875>

1	0	0	0	0	1	1	1	0	0	0	0	0	0	1
1	1	0	0	0	0	0	0	1	1	1	0	0	1	0

Problem Formulation

Learning a Distribution

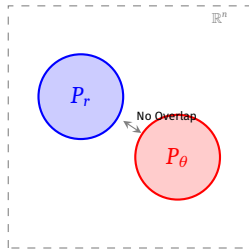
What does it mean to learn a probability distribution?

Possibly, learn a probability density function!

- ⚡ We define a parametric family of densities P_θ such that $\theta \in \mathbb{R}^d$ for d -dimensions and find the one that maximizes the likelihood on our data.
- ⚡ For a given data samples $\{x^{(i)}\}_{i=1}^m$, we would want

$$\max_{\theta \in \mathbb{R}^d} \frac{1}{m} \sum_{i=1}^m \log P_\theta(x^{(i)})$$

If there is a real data distribution, say, P_r , we want to reduce the distance between P_r and P_θ . How to define distance? (We describe later).



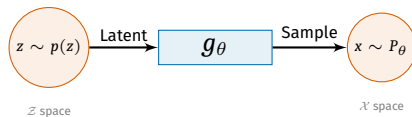
If P_r is simple/low-dimensional, P_θ might not exist. Overlap between P_θ and P_r might be zero or negligible.

Note: In generative models, we add noise to make the maximum likelihood approach work (so that some overlap may be found).

Problem Formulation

Generator Mapping

As P_r may not exist, we can define the random variable Z with a fixed distribution $p(z)$ and pass it through a parametric function $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ that can generate samples following P_θ .



| We already know g_θ is a generator from GAN.

Measuring Closeness

But how close is the model distribution to the real distribution?

- ⚡ Closeness can be measured using a distance or divergence ρ .
- ⚡ Since GANs generate a sequence of distributions (evolved during training epochs), we are interested in whether the sequence of distributions P_t converges (iff there is P_∞ such that $\rho(P_t, P_\infty) \rightarrow 0$).

Desired Properties of ρ

- 1 ρ should be weak so that it is easier to find a mapping $\theta \mapsto P_\theta$.
- 2 ρ should lead to a continuous mapping.
- 3 ρ should lead to a continuous loss function.

Measure of Closeness

Metrics vs. Divergence

Distance metrics obey the principles of non-negativity, identity, symmetry, and triangle inequality. But divergence is a weaker concept that doesn't follow symmetry and the triangle inequality.

The Foundation

- ⚡ Define closeness in metric set $\mathcal{X}$ (e.g., images $[0, 1]^d$).
- ⚡ Σ is the set of all Borel subsets of $\mathcal{X}$.
- ⚡ **Borel subsets:** Standard collection of "well-behaved" sets where you define probabilities. (Refer to Probability 101).



Measurable Space (X, Σ)

A Borel subset is any set that belongs to the Borel σ -algebra, the standard collection of "well-behaved" sets used in measure theory and probability. We want to assign probabilities or measures to subsets of $\mathbb{R}$. We cannot assign probabilities to just any kind of set; we need a reasonable set. A Borel subset is a reasonable subset. Borel sets are the sets you can actually

Formality: σ -algebra & Measure

σ -algebra: Nonempty collection Σ of subsets of X closed under complement, countable unions, and countable intersections.

Measure Theory: Generalization of geometric measures (length, area, probability). A function $\mu : \Sigma \rightarrow \mathbb{R}$ is a **measure** if:

- 1 $\mu(\emptyset) = 0$
- 2 $\mu(E) \geq 0 \quad \forall E \in \Sigma$
- 3 $\mu(\bigcup_{k=1}^{\infty} E_k) = \sum_{k=1}^{\infty} \mu(E_k)$

Distance and Divergences as Measures of Closeness

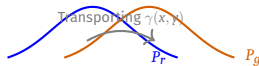
Total Variation (TV)

$$\delta(P_r, P_g) = \sup_{A \in \Sigma} [P_r(A) - P_g(A)]$$

Note: The **sup** finds the least upper bound. It's the "maximum" generalized to cases where a max might not exist. E.g., no max in $(0, 1)$ but sup is 1.

Kullback-Leibler (KL)

$KL(P_r || P_g) = \int \log \left(\frac{P_r(x)}{P_g(x)} \right) P_r(x) d\mu(x)$ Assumes *absolute continuity*. KL is asymmetric and can be ∞ if $P_g(x) = 0$ while $P_r(x) > 0$.



Jensen-Shannon (JS)

$JS(P_r, P_g) = KL(P_r || P_m) + KL(P_g || P_m)$ Where $P_m = (P_r + P_g)/2$. Symmetrical and always defined, as we can choose $\mu = P_m$.

Wasserstein (EM Distance)

$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$ Π is the set of joint distributions $\gamma(x, y)$ with marginals P_r, P_g . Essentially, you can say that $\gamma(x, y)$ is how much mass can be transported from x to y in order to transform P_r to P_g . The EM distance is the cost of this **optimal transport** plan. We will revisit this later separately.

Inf infimum is the opposite of supremum, it is the greatest lower bound. For example, in $(0, 1]$, 0 is the **inf**.

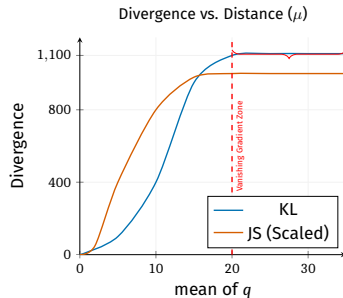
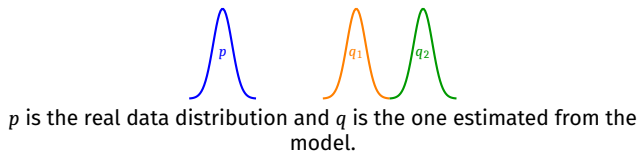
Not all measures are the same: Vanishing Gradients in KL and JS Divergence

The Gradient Issue

When the distributions p and q are disjoint (have no overlap), the divergences saturate.

- ⚡ As the mean of q increases, the divergence increases but flattens.
- ⚡ The gradient eventually **diminishes** to zero.
- ⚡ **Result:** The generator learns nothing from gradient descent.

$$D_{KL}(P||Q) = \sum_{x=1}^N P(x) \log \frac{P(x)}{Q(x)}$$

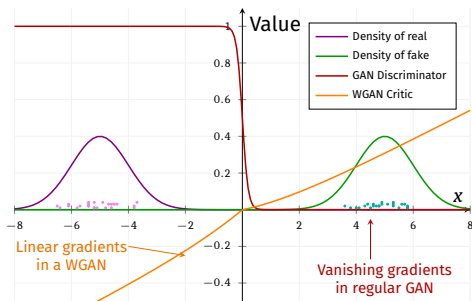


Traditional divergences do not provide a smooth, linear gradient for non-overlapping distributions, leading to training instability in GANs.

JS Divergence in the Original GAN

Minimizing the GAN objective function with an optimal discriminator is equivalent to minimizing the JS-divergence.

Using Wasserstein Distance with GAN



In WGAN, the discriminator is replaced by a *critic* f_w , which **scores** real and fake samples rather than classifying them.

Direct W is Intractable

Further, the Wasserstein Distance shown earlier is intractable as the set $\Pi(P_r, P_g)$ is infinitely large, and high-dimensional data makes γ , and the infimum requires solving a Combinatorial Optimization Problem, which is very difficult. Hence, instead, we opt for a dual version of Wasserstein distance as per **Kantorovic-Rubinstein duality**:

$$W(P_r, P_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_\theta}[f(x)]$$

With this dual, instead of optimizing over an infinite set of transport plans γ , we optimize over a neural network f (the Critic) — a finite-dimensional parameterization of 1-Lipschitz functions. The dual problem is differentiable (neural networks are differentiable), so we can use gradient descent to train the Critic.

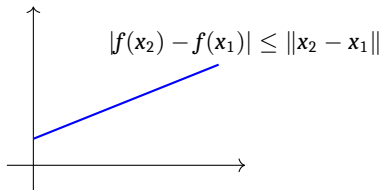
1-Lipschitz Function

Definition (1-Lipschitz)

A function $f : \mathcal{X} \rightarrow \mathbb{R}$ is 1-Lipschitz if

$$\forall x_1, x_2 \in \mathcal{X} : |f(x_2) - f(x_1)| \leq \|x_2 - x_1\|.$$

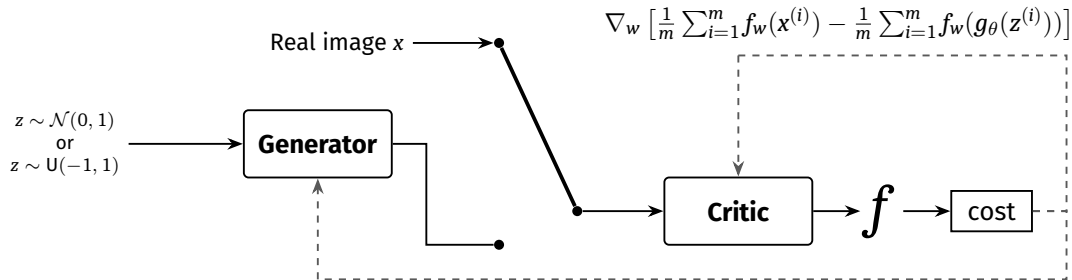
- ⚡ Output changes **no faster** than input changes.
- ⚡ If differentiable: $\|\nabla f(x)\| \leq 1$ everywhere.
- ⚡ Ensures the Critic f is **stable and well-behaved**.
- ⚡ Required for the **Kantorovich–Rubinstein duality** in WGAN.



WGAN Architecture

We can also go for K-Lipschitz, then we get $K\hat{W}(P_r, P_g)$. Then we can have parametrized family of functions $\{f_w\}_{w \in \mathcal{W}}$ that are K-Lipschitz and we can think of solving

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim P_r} [f_w(x)] - \mathbb{E}_{z \sim p(z)} [f_w(g_\theta(z))]$$



$$\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$$

WGAN Algorithm

Algorithm

Require: α (LR), c (clipping), m (batch), n_{critic} (no. of iterations of the critic per generator iteration)

Require: w_0 (initial critic), θ_0 (initial generator's parameter)

while θ not converged **do**

for $t = 0, \dots, n_{\text{critic}}$ **do**

 Sample $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$ a batch from the real data;

 Sample $\{z^{(i)}\}_{i=1}^m \sim p(z)$ a batch of prior samples;

$g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))];$

$w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w);$

$w \leftarrow \text{clip}(w, -c, c) \leftarrow \text{Lipschitz};$

 Sample $\{z^{(i)}\}_{i=1}^m \sim p(z);$

$g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}));$

$\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta);$



All experiments used $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Reference

- 1 Arjovsky, Martin, Soumith Chintala, and Léon Bottou. "Wasserstein GAN." arXiv preprint arXiv:1701.07875 (2017). <https://arxiv.org/abs/1701.07875>

Data Privacy and GANs

0	0	0	1	0	1	1	0	0	0	0	1	0	1	0	0	0	0	0
1	0	0	0	0	0	1	0	0	0	1	1	0	1	1	0	0	1	0
1	0	1	0	0	1	0	0	1	0	1	1	1	0	0	1	0	0	0

Privacy issues with GANs

GANs are trained to **model the real data distribution**.

But in doing so, they can:

- ⚡ **Memorize** training samples (faces, medical data, text)
- ⚡ **Reconstruct** exact or near-exact training data
- ⚡ Leak **sensitive attributes**: gender, disease, location, ID
- ⚡ Comply with no privacy by default

GANs **implicitly store information** about training data in the generator and discriminator weights.

Examples of leakage:

- ⚡ Face GANs: reproduce exact faces from the training set
- ⚡ Medical GANs: reveal unique patient biomarkers
- ⚡ Text GANs: regurgitate private sentences

Membership Inference Attacks:

Given a sample x , the attacker asks: Was x in the training set?

How it works:

- ⚡ Use discriminator confidence $D(x)$
- ⚡ Training samples have **unusually high/low confidence**
- ⚡ Attacker trains a classifier to detect this

Privacy Issues with GANs

Modeling vs. Memorization

GANs are trained to **model the real data distribution**.

But in doing so, they can:

- ⚡ **Memorize** training samples (faces, medical data, text)
- ⚡ **Reconstruct** exact or near-exact training data
- ⚡ Leak **sensitive attributes**: gender, disease, location, ID
- ⚡ Comply with no privacy by default

GANs **implicitly store information** about training data in the generator and discriminator weights.

Examples of Leakage:

- ⚡ **Face GANs**: reproduce exact faces
- ⚡ **Medical GANs**: reveal unique biomarkers
- ⚡ **Text GANs**: regurgitate private sentences

Membership Inference Attacks

Given a sample x , the attacker asks: *Was x in the training set?*



How it works:

- ⚡ Use discriminator confidence $D(x)$
- ⚡ Training samples have **unusually high/low confidence**
- ⚡ Attacker trains a classifier to detect this

Application of GANs in Industrial Settings

Some Examples:

- 1 Manufacturing Defect Detection & Quality Control
- 2 Synthetic Data Augmentation
- 3 Digital Twins and Virtual Prototyping
- 4 3D Modeling and Design: 2D $\rightarrow$ 3D
- 5 Image Enhancement and Restoration
- 6 Denoising

Code

[https://github.com/rahulbhadani/CPE487587_SP26/
blob/master/Code/Chapter_06_Generative_
Adversarial_Network.ipynb](https://github.com/rahulbhadani/CPE487587_SP26/blob/master/Code/Chapter_06_Generative_Adversarial_Network.ipynb)

References



Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio (2013)
Generative Adversarial Networks
Advances in Neural Information Processing Systems.



Isola, Phillip, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. "Image-to-image translation with conditional adversarial networks." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1125-1134. 2017. <https://arxiv.org/pdf/1611.07004>



Mirza, Mehdi, and Simon Osindero. "Conditional generative adversarial nets." arXiv preprint arXiv:1411.1784 (2014). <https://arxiv.org/pdf/1411.1784>



Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." arXiv preprint arXiv:1511.06434 (2015). <https://arxiv.org/pdf/1511.06434>



Zhu, Jun-Yan, Taesung Park, Phillip Isola, and Alexei A. Efros. "Unpaired image-to-image translation using cycle-consistent adversarial networks." In Proceedings of the IEEE International Conference on Computer Vision, pp. 2223-2232. 2017. <https://arxiv.org/pdf/1703.10593>



Arjovsky, Martin, Soumith Chintala, and Léon Bottou. "Wasserstein GAN." arXiv preprint arXiv:1701.07875 (2017). <https://arxiv.org/abs/1701.07875>

The End